

Technical Analysis Report

Subject: Evaluation of the Provided Trading Platform Codebase Against Project Requirements

Prepared By: Phestone

Role: Lead Developer / Technical Architect

Executive Summary

After thoroughly reviewing the provided project, I can confidently state that the application is **well developed for its intended purpose**. The codebase demonstrates good software engineering practices, organized architecture, clear separation of concerns, and solid implementation of trading strategies.

However, after comparing the implementation with the project requirements that were originally discussed, I have concluded that **this project does not meet the business requirements of the requested platform**.

The main reason is that the application is **not an independent binary trading platform**.

Instead, it is a **client-side trading terminal that connects existing Deriv accounts to automate trading through the Deriv WebSocket API**.

In simple terms:

This application helps existing Deriv users automate their trading.

It does **not** create or operate a trading business of its own.

How the Application Actually Works

The entire system is designed around the assumption that every user already owns a Deriv trading account.

The application flow is as follows:

User

↓

Create/Login Account

↓

Enter Personal Deriv API Token

↓

Application connects directly to Deriv

↓

Receives Market Data

↓

Executes Trades on Deriv

↓

Displays Results

The application itself never becomes responsible for holding money or operating a trading platform.

Instead, it acts as a bridge between the user and Deriv.

Responsibilities Managed by Deriv

Once connected, nearly every important business operation is performed by Deriv rather than by the application itself.

These include:

- User trading account
- Wallet balance
- Deposits
- Withdrawals
- Trade execution
- Account verification
- Live market prices
- Profit calculation
- Contract settlement

This means the application does not actually own or manage any financial infrastructure.

Comparison Against Our Project Requirements

Our project requirement is fundamentally different.

The requested platform is intended to become an independent trading business where every user belongs directly to our platform.

Below is a comparison between the requested system and the provided implementation.

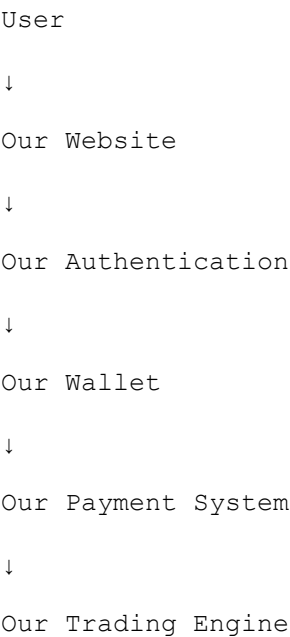
Required Feature	Status
User Registration	☒ Available
Independent User Accounts	☒ Not Available
Internal Wallet System	☒ Missing
Deposit System (M-Pesa etc.)	☒ Missing
Withdrawal System	☒ Missing
Payment Gateway Integration	☒ Missing
Transaction History	☒ Missing
User Ledger	☒ Missing
Referral System	☒ Missing
Admin Dashboard	☒ Missing
User Management	☒ Missing
KYC Verification	☒ Missing
Internal Trading Engine	☒ Missing
Internal Trade Settlement	☒ Missing
Revenue Management	☒ Missing

The Core Difference

The biggest difference is ownership.

The provided project depends entirely on Deriv.

Our intended platform should instead function like this:



↓

Our Database

↓

Our Administration Panel

↓

Trade Settlement

↓

Withdrawals

In this architecture, every user belongs to us, all funds pass through our system, and every trade is executed and recorded by our own backend.

The provided project does not implement this model.

Why This Is Not Suitable For Our Business

One of the key business objectives discussed was that users should simply visit our website, create an account, deposit funds, and begin trading immediately.

The provided project cannot support that workflow.

Instead, every user must first:

- Create a Deriv account
- Deposit funds into Deriv
- Generate a Deriv API Token
- Connect their token to the application

Without a Deriv account, the application cannot function.

Therefore, users would effectively become Deriv customers rather than customers of our platform.

This is fundamentally different from the intended business model.

Existing Strengths of the Project

Although the application does not meet the project requirements, it contains several well implemented technical components that deserve recognition.

These include:

- Clean React architecture
- Organized component structure
- Real-time WebSocket communication
- EMA/RSI trading strategy implementation
- Straddle strategy implementation
- Risk management features
- Daily loss protection
- Strategy state management
- Market data visualization
- Firebase Authentication integration

These features demonstrate solid frontend engineering and trading automation knowledge.

Missing Backend Infrastructure

To become a complete standalone binary trading platform, the following backend systems would still need to be designed and implemented.

Authentication Service

- User registration
 - Login
 - Session management
 - Password recovery
 - User roles
-

Wallet System

- User balances
 - Credits
 - Debits
 - Wallet history
 - Ledger accounting
-

Payment System

- M-Pesa integration
 - Card payments
 - Deposit verification
 - Withdrawal processing
 - Payment reconciliation
-

Trading Engine

- Market price processing
 - Trade execution
 - Binary settlement
 - Profit calculations
 - Trade history
-

Administration Panel

- User management
 - Wallet adjustments
 - Deposit approvals
 - Withdrawal approvals
 - Reports
 - Analytics
 - Platform settings
-

Security

- Fraud detection
 - Rate limiting
 - Audit logs
 - Encryption
 - Activity monitoring
 - IP logging
-

Notification System

- Deposit notifications
 - Withdrawal notifications
 - Trade notifications
 - Email alerts
 - Push notifications
-

Reporting

- Revenue reports
- User statistics
- Trading statistics
- Daily profit reports
- Financial summaries

Estimated Development Scope

Based on the current requirements, building the requested platform is significantly larger than modifying the existing project.

This would require designing an entirely new backend responsible for:

- Managing users
- Managing funds
- Processing payments
- Executing trades
- Recording financial transactions
- Managing platform administration

The provided project could potentially serve as a useful reference for trading interface design and strategy implementation, but it should **not** be considered a complete foundation for the intended business.

Final Conclusion

From a software engineering perspective, the supplied application is a competent trading terminal designed to automate trading through Deriv.

However, after comparing it with the requested business objectives, I conclude that it does **not** satisfy the requirements for an independent binary trading platform.

The primary limitation is that it relies entirely on Deriv's infrastructure for trading, wallets, payments, and account management.

Our intended platform requires complete ownership of the user lifecycle, financial systems, trading engine, and administrative controls.

For that reason, the recommended approach is to design a dedicated backend architecture tailored specifically to our business requirements rather than attempting to adapt the current implementation into a standalone trading platform.